

Spatial Mental Modeling from Limited Views

Reproducing ViperGPT without Codex:
a Prompt-Specification Gap and a Depth-Grounded API

A Dissertation submitted in partial fulfilment
of the requirements for the degree of

Master of Technology
in
Computer Science and Engineering
(Information Security and Privacy)

submitted by

MD Humayun

under the supervision of

Dr. M. A. Zaveri

Department of Computer Science and Engineering
Sardar Vallabhbhai National Institute of Technology
Surat, Gujarat, India

2026

Abstract

ViperGPT answers a visual query by prompting a code-generating language model to emit a Python program over pretrained vision modules, and then executing that program. Its published generator, OpenAI Codex, has since been withdrawn, so any present-day reproduction must substitute a different model. This dissertation reports what that substitution reveals.

Rebuilding the pipeline with an open-weights generator produces an accuracy of 0.00 on RefCOCO referring-expression comprehension. The failure is not a crash: 332 of 500 programs execute to completion and return a natural-language string where a bounding box is required. The released API specification never states the return type a grounding program must produce, and five of its seven worked examples declare `str`. Stating the contract explicitly raises accuracy to 39.20. The effect is independent of model scale — it appears at 1.5B, 7B and 32B parameters and is repaired at each — and scaling the generator twentyfold buys approximately six accuracy points where stating the contract buys thirty-three to forty-one.

A second finding follows. Across those three model sizes, non-spatial accuracy rises from 33.65 to 48.82 while spatial accuracy remains flat at 34.95, 34.26 and 34.95, on identical images with identical perception modules. A larger generator writes better programs for everything except spatial relations. The bottleneck is the interface: the API exposes a single scalar for depth, from which inter-object ordering cannot be expressed, and the prompt directs the generator away from geometry.

The dissertation therefore extends the API with depth-ordering primitives and a corrected depth backend. Offered without qualification these improve depth-word queries from 4.26 to 19.15 but degrade non-spatial accuracy by ten points: given a new primitive, the generator applies it indiscriminately. Adding three sentences that delimit when the primitives apply improves every subset of both datasets, raising depth queries to 21.28 and overall RefCOCO accuracy to 42.00 (42.07 ± 0.61 over three seeds against 36.07 ± 1.75 for the contract alone), while reducing the seed-to-seed variance of spatial queries by a factor of 4.6.

Two results large enough to be attractive were rejected after control experiments. Capping the detector’s candidate set raises accuracy by fifteen points, but a condition containing no depth primitives gains almost as much, so the improvement belongs to the candidate set rather than to the method. Separately, a depth-sign correction that a synthetic validation had rejected was reinstated after testing against ground-truth annotations, where the uncorrected signal scored below chance.

Keywords: visual reasoning, program synthesis, referring expression comprehension

sion, reproducibility, spatial grounding, vision–language models.

Contents

Abstract	ii
1 Introduction	1
1.1 Background	1
1.2 ViperGPT	1
1.3 Problem statement	2
1.4 Research questions	2
1.5 Approach	3
1.6 Contributions	3
1.7 Scope	4
2 Literature Review	5
2.1 Compositional visual reasoning	5
2.2 Program synthesis with language models	5
2.2.1 The generator as an undocumented dependency	5
2.3 Referring expression comprehension	6
2.3.1 The spatial component	6
2.4 Spatial reasoning in vision–language models	6
2.5 Positioning	7
3 Problem Formulation	8
3.1 Formal setting	8
3.2 Problem 1: the generator is unavailable	8
3.3 Problem 2: the specification is incomplete	8
3.4 Problem 3: the interface cannot express depth relations	9
3.5 Problem 4: whether an extended interface helps	9
3.6 Criteria for a claim	10
4 System Architecture and Methodology	11
4.1 Architecture	11
4.1.1 The stage boundary	12
4.2 Departures from the published design	12
4.3 Prompt specification	13
4.3.1 C1 — the released specification	13
4.3.2 C2 — an explicit output contract	14
4.3.3 C3 — depth primitives	15
4.3.4 C4 — routing	16

4.4	Evaluation protocol	17
4.4.1	Metric	17
4.4.2	Handling of invalid predictions	17
4.4.3	Data	17
5	Implementation	18
5.1	Environment	18
5.2	Building GLIP for the Hopper architecture	18
5.2.1	Independent verification	19
5.3	The detector wrapper	19
5.4	The depth backend	20
5.5	The depth primitives	20
5.5.1	Unit tests	21
5.6	Execution and scoring	22
6	Experimental Setup	23
6.1	Detector threshold	23
7	Results	25
7.1	Main result	25
7.2	Comparison with the published figures	26
7.3	Why the released prompt scores zero	26
7.3.1	A second effect: output format	28
7.3.2	Static analysis	28
7.4	Generator scale	28
7.5	Spatial breakdown	29
7.6	API adoption	30
7.7	Providing a capability is not sufficient	30
7.8	Seed variance	31
7.9	Robustness of the metric	33
8	Rejected Hypotheses and Controls	35
8.1	Rejection 1: the collapse hypothesis	35
8.2	Rejection 2: RefCOCO+ as a clean control	35
8.3	Rejection 3: the primitives alone	36
8.4	A fifteen-point result, disqualified	36
8.5	A sign error that a synthetic test endorsed	37
9	Deviations from the Published Configuration	40
9.1	Undocumented at the time of writing	41

10 Limitations	42
11 Conclusion and Future Work	43
11.1 Conclusion	43
11.2 Future work	44

List of Figures

4.1	Reproduced system architecture	11
4.2	Published design against this implementation	12
6.1	Detector threshold trade-off	24
7.1	Main result	25
7.2	Reproduction against published figures	26
7.3	Reproduction with the perception ceiling	27
7.4	Failure decomposition	27
7.5	Accuracy against generator scale	29
7.6	API adoption across conditions	31
7.7	Qualitative comparison	32
7.8	Seed variance and stability	32
7.9	IoU threshold sensitivity	33
7.10	IoU distribution	34
8.1	The control that disqualified a fifteen-point result	37
8.2	Achieved against achievable	38
8.3	Depth sign validation	38

List of Tables

3.1	Return types declared by the worked examples in the released specification (<code>prompts/api.prompt</code> , 7,645 bytes). Line numbers refer to that file.	9
4.1	The four prompt conditions.	13
5.1	Execution environments. The GLIP fork requires an older PyTorch than the generator; the pipeline is split at the <code>programs.jsonl</code> boundary as a result.	18
6.1	Experiments. Generation and execution are separate jobs; identifiers are scheduler references retained for provenance.	23
6.2	Detector threshold sweep, held-out sample of 200. The selected value of 0.2 lies within one point of the optimum for detection.	23
7.1	Accuracy at $\text{IoU} \geq 0.5$. Qwen2.5-Coder-7B, greedy decoding, $n = 500$ per cell, testA split. C1 is the released prompt; C4 adds both the depth primitives and the rule stating when they apply.	25
7.2	What each of the 500 samples returned, RefCOCO/testA.	26
7.3	Static analysis of generated programs, $n = 500$ per cell. “Task-invalid” denotes a program whose <code>return</code> yields a string rather than a patch; “spatial dropped” denotes a spatial query whose program contains no positional operation at all.	28
7.4	Generator scale, condition C2, greedy, RefCOCO/testA, $n = 500$. . .	28
7.5	The two spatial subsets are not the same kind of query.	30
7.6	API calls across 500 generated programs, RefCOCO/testA.	30
7.7	Seed variance, RefCOCO/testA, $T = 0.7$, seeds 1–3, $n = 500$	31
8.1	Candidate cap applied to C4 programs, RefCOCO/testA.	36
8.2	The control required by criterion C-1. C2 has no depth reasoning anywhere in the pipeline.	36
8.3	Depth sign validated against ground truth, no detector involved, $n = 25$. .	38
9.1	Deviation register.	40

Chapter 1

Introduction

1.1 Background

A question asked about an image is rarely atomic. “*The man on the right*” requires locating every man in the scene and then ordering them; “*the cup behind the bottle*” requires locating two object classes and then comparing their distance from the camera. Monolithic vision–language models answer such queries in a single forward pass, which makes the intermediate reasoning inaccessible: when the answer is wrong, there is no step to point at.

An alternative treats the query as a *program to be written*. A large language model is shown a specification of an API over pretrained vision modules and asked to emit code that composes those modules; the code is then executed against the image. Neural Module Networks introduced composition over learned modules [1], and capable code-generation models made it possible to synthesise the composition directly in a general-purpose language. VisProg [4] and ViperGPT [18] are the prominent instances.

The appeal is threefold. The system is *training-free*: no gradient step is taken, and new capabilities are added by extending the API rather than by retraining. It is *compositional*: a query never seen during training can be answered if its parts can be. And it is *interpretable* in a strong sense, since the intermediate reasoning is a Python program a human can read and a machine can execute step by step.

1.2 ViperGPT

ViperGPT formalises this as a two-stage pipeline. A code generator π maps a query q to a program z , and an execution engine ϕ runs that program against an image x :

$$z = \pi(q), \quad r = \phi(x, z).$$

The generator is prompted with an API specification describing an `ImagePatch` class and functions over it — `find` for open-vocabulary detection, `verify_property` for attributes, `simple_query` for visual question answering, `compute_depth` for relative depth, and others. The generator never sees the image; it sees only the query and the API. Execution binds the API to pretrained models: GLIP [10] for detection,

X-VLM [21] for attributes, BLIP-2 [9] for question answering, MiDaS [15] for depth.

On referring-expression comprehension the paper reports 72.0 on RefCOCO and 67.0 on RefCOCO+ (testA), against 55.0 and 52.2 for GLIP used directly and 58.6 and 60.5 for ReCLIP [16].

1.3 Problem statement

The generator used in that work, `code-davinci-002`, has been withdrawn. Any attempt to reproduce or build on ViperGPT today must substitute a different generator, and no faithful option exists. This is not a peculiarity of one paper: a published system whose central component is reachable only through a proprietary API is reproducible only while that API is maintained.

Substitution raises a question the original work had no reason to ask. *Which parts of the reported behaviour belong to the method, and which belonged to Codex?* An API specification written for one model is an interface contract; if part of that contract was carried implicitly by the model’s own inference rather than by the text, the omission is invisible until the model changes.

A second question follows from the API itself. Of the operations ViperGPT exposes, exactly one concerns three-dimensional structure: `compute_depth()`, returning a single scalar. From one scalar per object it is possible to express “*A is behind B*”. It is not possible to express “*the third-closest person*”, “*between A and B in depth*”, or physical size. Meanwhile the prompt directs the generator to resolve *left*, *right*, *up* and *down* using base Python on patch coordinates. Whether this suffices for the spatial language referring expressions actually contain is an empirical question that an aggregate number per dataset does not answer.

1.4 Research questions

RQ1

When Codex is replaced by an open-weights code generator, what happens on referring-expression comprehension, and how much of any shortfall is attributable to the released prompt rather than to the substituted model?

RQ2

Is that effect a property of the prompt or of the generator’s capacity — does it change with model scale?

RQ3

Does accuracy differ systematically by the *kind* of spatial language a query contains, and is any difference explained by the expressiveness of the API?

RQ4

Does extending the API with depth-ordering primitives improve accuracy on the queries that need them, without degrading those that do not?

1.5 Approach

The work proceeds in three milestones, each stated as a hypothesis and tested.

Milestone 1 reproduces the generation stage alone. Programs are synthesised for 500 RefCOCO and 500 RefCOCO+ queries and analysed *statically* — parsed, not executed — so the generator can be studied before the perception stack exists. The initial hypothesis, that spatial queries would collapse into a single opaque `simple_query` call, was rejected; the analysis instead revealed a return-type failure affecting 98.8% of programs.

Milestone 2 builds the execution stack and measures end-to-end accuracy. This required compiling the GLIP fork for the Hopper architecture, which the published environment cannot target, and substituting CLIP [14] for X-VLM. A hypothesis that RefCOCO+ would provide a clean control for spatial language was rejected: the gap widened rather than narrowed, and a manual audit established that the two datasets contain different *kinds* of spatial language.

Milestone 3 extends the API with depth-grounded primitives. A first form, offering the primitives without specifying when they apply, improved the target queries and degraded the others; the final form adds a routing rule and improves every subset measured.

1.6 Contributions

1. **A working reproduction with an open-weights generator**, on hardware the published environment cannot run on. The GLIP fork is rebuilt for compute capability 9.0 and verified independently of the full pipeline.
2. **Identification and quantification of a specification gap**. The released prompt does not state the required return type, and five of its seven worked examples declare `str`. With an open-weights generator, 98.8% of programs return a string where a box is required; executed, this yields 0.00. Stating the contract raises it to 39.20.
3. **Evidence that this is a property of the specification, not of capacity**. The failure occurs at 1.5B, 7B and 32B (95.8%, 98.8%, 87.6% task-invalid) and is repaired at every scale. Scaling twentyfold buys ≈ 6 points; the contract buys 33–41.

4. **Evidence that spatial accuracy does not respond to scale.** Non-spatial accuracy rises $33.65 \rightarrow 48.82$ across the three models while spatial accuracy stays at 34.95/34.26/34.95 on identical images and modules.
5. **A depth-grounded API extension** — `depth_order`, `is_behind`, `is_in_front_of`, `is_between_3d`, `distance_3d` — with a corrected depth backend. The generator adopts it, invoking `depth_order` 119 times across 500 programs where it invokes the original `compute_depth` 29 times.
6. **The finding that providing a capability is insufficient without specifying when it applies.** The primitives alone move depth queries $4.26 \rightarrow 19.15$ and cost ten points of non-spatial accuracy; three sentences delimiting their scope improve every subset, reaching 21.28 on depth queries and 42.00 overall.
7. **Two negative results established by control rather than omitted:** a 15-point apparent gain from capping the candidate set, shown to arise from the candidate set rather than from depth reasoning; and a depth-sign error that a synthetic validation had endorsed, identified by testing against ground truth.

1.7 Scope

Evaluation is restricted to referring-expression comprehension on RefCOCO and RefCOCO+ — Table 1 of the original paper. GQA and NExT-QA are excluded for storage and compute; OK-VQA depends on GPT-3, likewise withdrawn. No cross-task claim is made.

The reproduction is not faithful and is not presented as such: the final configuration reaches 42.00 against a reported 72.0, and the responsible deviations are enumerated in Chapter 9. What is reproduced is the *method*; what is measured is what happens to it when its generator is replaced.

Depth is claimed only as *ordering*. The datasets provide no camera intrinsics, so metric distances would rest on an assumed field of view; the ordering primitives require only monotonicity in true distance, which relative depth provides once its sign is correct.

Chapter 2

Literature Review

2.1 Compositional visual reasoning

The observation that a visual question decomposes into simpler operations predates large language models. Neural Module Networks [1] parse a question into a layout of learned modules and assemble a network per instance. The modules are trained jointly, so the approach inherits both the strengths of end-to-end learning and its costs: a new capability requires retraining, and the layout parser must itself be learned.

Later work refined the parsing rather than the principle. Stack-NMN [5] makes the layout differentiable; Neural-Symbolic VQA [19] separates perception from a symbolic executor over a structured scene representation. The recurring difficulty is that the program space must be fixed in advance, because it is the space over which the parser is trained.

2.2 Program synthesis with language models

Code-generating models change that constraint. If a model can write Python from a specification, the program space is the language itself, and adding a capability becomes a matter of documenting a function rather than retraining a parser.

VisProg [4] generates programs in a small domain-specific language over a fixed module set. ViperGPT [18] generates ordinary Python, supplies an API specification as the prompt, and executes the result with `exec`. CodeVQA [17] applies the idea to question answering with a smaller module set; Chameleon [12] generalises to tool sequences beyond vision.

These systems share three properties: they are training-free, their intermediate representation is inspectable, and their behaviour is determined by two artefacts that receive little attention in evaluation — the API surface and the prompt describing it. This dissertation studies the second pair.

2.2.1 The generator as an undocumented dependency

ViperGPT uses Codex; VisProg uses GPT-3; CodeVQA uses Codex. All three models have been withdrawn. The published prompts remain, but the model that

interpreted them does not, and there is no way to establish after the fact which parts of a system’s behaviour were specified by the prompt and which were supplied by the model.

Related concerns appear in the reproducibility literature. Kapoor and Narayanan [8] document how evaluation flaws propagate through published results; Bell and Kampman [2] survey reproducibility in AI research more broadly. The failure mode considered here — a system whose central component becomes permanently unavailable — is a consequence of building on closed APIs, and this work is a case study in what such a dependency conceals.

2.3 Referring expression comprehension

Referring expression comprehension asks a model to localise the object described by a phrase. RefCOCO and RefCOCO+ [20] are the standard benchmarks, collected on COCO images. They differ in one deliberate respect: RefCOCO permits location words, RefCOCO+ forbids them, so its expressions describe objects by appearance rather than position.

Strong baselines follow two routes. Detection-based methods — MDETR [6], GLIP [10], OWL-ViT [13] — pretrain for grounded detection directly. Composition-based methods such as ReCLIP [16] decompose the expression and score candidate regions with CLIP without task-specific training. ViperGPT belongs to a third category: it neither trains nor scores directly, but writes a program that calls a detector.

2.3.1 The spatial component

An aspect of RefCOCO that bears on this work is that its expressions contain several distinct kinds of spatial language. *Left, right, top* and *bottom* are viewer-relative and resolvable in the image plane. *Closest, farthest, behind* and *in front of* concern distance from the camera and are not. Because RefCOCO+ excludes location words but not distance words, the two datasets differ in the *composition* of their spatial subsets, not merely in quantity. Aggregate accuracy cannot distinguish these cases; Chapter 7 shows they behave differently.

2.4 Spatial reasoning in vision–language models

Recent work reports that vision–language models handle spatial relations poorly. SpatialVLM [3] attributes this to a lack of 3D supervision and synthesises spatial

question-answering data at scale. Benchmarks such as What’s Up [7] and VSR [11] isolate spatial relations and find large models near chance on some of them.

That literature concerns models trained end to end, where the deficit is a property of the learned representation. In a program-synthesis system the situation differs: spatial relations are resolved by code, so the deficit may lie in the model, in the modules, or in the interface between them. Distinguishing these can be done experimentally, and Chapter 7 does so by varying model scale while holding the interface fixed.

2.5 Positioning

This dissertation is a reproduction study with an extension. It differs from the above in three respects. It treats the API specification as an object of study rather than as infrastructure. It separates the contribution of the generator from that of the interface by varying model scale — a control available in a training-free system and rarely reported. And it reports two negative results established by control rather than omitting them.

Chapter 3

Problem Formulation

3.1 Formal setting

Let q be a query, x an image, and $\mathcal{A}$ an API consisting of functions over image regions together with a textual specification $s(\mathcal{A})$. A generator π maps specification and query to a program, and an engine ϕ evaluates it:

$$z = \pi(s(\mathcal{A}), q), \quad r = \phi(x, z; \mathcal{A}).$$

For grounding the required output is a region, so a correct program satisfies $r \in \mathcal{B}$. Accuracy is measured by intersection over union against the ground-truth box b^* :

$$\text{IoU}(r, b^*) = \frac{|r \cap b^*|}{|r \cup b^*|}, \quad \text{correct} \iff \text{IoU}(r, b^*) \geq \tau, \quad \tau = 0.5.$$

Three properties matter. The generator never observes x . Nothing in the type system enforces $r \in \mathcal{B}$, because z is ordinary Python and ϕ is `exec`. And $s(\mathcal{A})$ is prose, not a schema, so any constraint it fails to state is unenforced.

3.2 Problem 1: the generator is unavailable

ViperGPT instantiates π as Codex, which has been withdrawn; reproduction requires π' . The substitution changes the entity that interprets $s(\mathcal{A})$. If some constraint c holds of the programs Codex produced, two explanations exist: c is stated in $s(\mathcal{A})$, or Codex inferred it. These are indistinguishable while the model is available and distinguished the moment it is replaced.

3.3 Problem 2: the specification is incomplete

The released specification is task-agnostic — the same text serves question answering and grounding. It contains seven worked examples, whose declared return types are distributed as in Table 3.1.

Five of seven return a string. One returns a list. Exactly one returns the type a grounding program requires, and it sits inside the docstring of `compute_depth`,

Table 3.1: Return types declared by the worked examples in the released specification (prompts/api.prompt, 7,645 bytes). Line numbers refer to that file.

Example documents	Declared return type	Line
find	List[ImagePatch]	44
exists	str	55
verify_property	str	68
best_text_match	str	79
simple_query	str	92
compute_depth	ImagePatch	104
llm_query	str	123

where a reader seeking the output contract has no reason to look. No sentence states that `execute_command` must return an `ImagePatch`.

3.4 Problem 3: the interface cannot express depth relations

Let $d(p)$ denote `compute_depth` for a patch p . The relation *a is behind b* is expressible as $d(a) > d(b)$. The following are not expressible without the program implementing the ordering logic itself:

- *the third-closest person* — a total order over a set;
- *the cup between the bottle and the plate in depth* — an interval test over three patches;
- *the larger of the two objects* — physical size, hence back-projection.

The specification compounds this. Among its three guidelines:

Use base Python (comparison, sorting) for basic logical operations, left-/right/up/down, math, etc.

This is correct for image-plane relations and cannot be correct for depth relations, because the image plane does not contain the information.

3.5 Problem 4: whether an extended interface helps

If the interface is the constraint, extending it should improve the affected queries. Two failure modes must be distinguished from success: the generator may ignore the new functions, as it largely ignores `compute_depth`; or it may apply them indiscriminately, improving the target queries at the expense of others.

3.6 Criteria for a claim

Because this is a reproduction study, the standard of evidence is fixed in advance.

C-1 Attribution

Any improvement must be shown to arise from the intervention rather than from a change in the evaluation. Where an intervention could plausibly help for an unrelated reason, a control condition without it is run.

C-2 Non-degradation

An improvement confined to a target subset at the cost of others is reported as a trade-off, not as a gain.

C-3 Stability

Where decoding is stochastic, results are reported as mean and standard deviation over three seeds.

C-4 Provenance

Every reported figure traces to a stored run summary, a configuration, and a scheduler job identifier.

C-5 Disclosure

Every departure from the published configuration is recorded with its reason and expected effect, whether or not it is believed to matter.

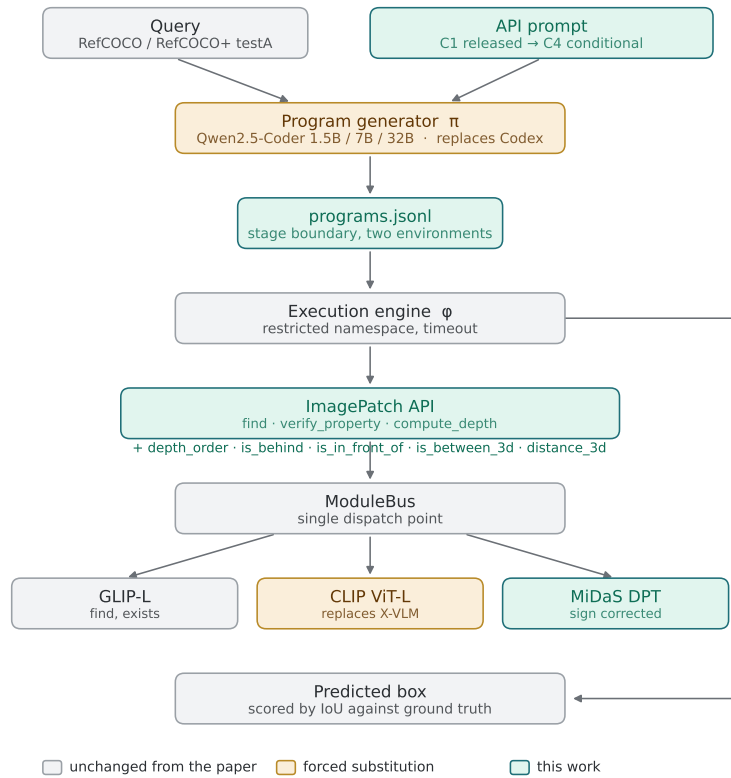
Criteria C-1 and C-2 are not decorative. Chapter 8 reports two interventions rejected under C-1, and one that satisfied its target subset but failed C-2 and was superseded.

Chapter 4

System Architecture and Methodology

4.1 Architecture

Figure 4.1 shows the reproduced pipeline. Colour marks provenance: components carried over unchanged, components substituted because the published choice is unavailable, and components introduced by this work.



MD Humayun · supervised by Dr. M.A. Zaveri · M.Tech CSE (Information Security and Privacy), SVNIT Surat

Figure 4.1: The reproduced pipeline. A query and an API specification are given to the program generator; the emitted program is written to disk, then executed against the image by an engine that binds the API to pretrained modules. The predicted region is scored by IoU.

4.1.1 The stage boundary

The published design runs generation and execution in one process. This implementation writes programs to `programs.jsonl` between the stages.

The immediate reason is a dependency conflict: the GLIP fork requires an older PyTorch than the code generator, and the two cannot share an interpreter. Two consequences follow, both useful. Prompt conditions are compared on *identical* programs, since the execution stage reads a fixed file. And the generation stage can be studied before the execution stage exists, which is what makes the static analysis of Section 7.3 possible.

4.2 Departures from the published design

	ViperGPT (Surís et al., ICCV 2023)		This work
Prompt	one task-agnostic API spec	→	four variants, C1 to C4
Generator π	Codex code-davinci-002	→	Qwen2.5-Coder, three sizes
Pipeline	single process, producer-consumer	→	two stages split at <code>programs.jsonl</code>
Depth API	<code>compute_depth()</code> → one scalar	→	<code>depth_order</code> , <code>is_behind</code> , <code>distance_3d</code> , ...
Depth backend	MiDaS, output used as returned	→	MiDaS, sign corrected and resampled
verify_property	X-VLM	→	CLIP ViT-L
simple_query / llm_query	BLIP-2-XXL / GPT-3	→	neutral fallback, out of scope
GLIP build	torch 1.13, CUDA 11.6	→	torch 2.1.2, CUDA 12.1, sm_90
Detection threshold	0.5	→	0.2, validated held out
Scoring	mean IoU; missing → mean box	→	acc@0.5 and mean IoU; missing → 0
Validation	seeds not stated	→	12 deviations, 3 seeds, 2 controls

Unchanged: the method itself — query → program → execute — the executor semantics, GLIP as detector, and the task.

MD Humayun · supervised by Dr. M.A. Zaveri · M.Tech CSE (Information Security and Privacy), SVNIT Surat

Figure 4.2: Component-by-component comparison. Amber marks a substitution forced by unavailability; teal marks a deliberate change. The unchanged elements are the method, the executor semantics, the detector, and the task.

Every departure carries an identifier D1–D12 and is documented in Chapter 9. The substitutions that most affect results are the generator (D1) and the attribute

model (D9).

4.3 Prompt specification

The experimental variable is the API specification. Four variants are defined; each is a strict superset of the previous one, so any difference in outcome is attributable to the added text. Model, modules, data and decoding are held fixed.

Table 4.1: The four prompt conditions.

	File	Bytes	Added
C1	api.prompt	7,645	— the released specification
C2	api_grounding.prompt	8,639	an explicit output contract, two grounding examples
C3	api_depth.prompt	11,485	five depth primitives with docstrings and three examples; removal of the instruction directing spatial relations to base Python
C4	api_depth_cond.prompt	12,680	three routing rules delimiting when the primitives apply, and two counter-examples

4.3.1 C1 — the released specification

The prompt declares an `ImagePatch` class with attributes `left`, `right`, `upper`, `lower`, `horizontal_center`, `vertical_center`, and methods `find`, `exists`, `verify_property`, `best_text_match`, `simple_query`, `compute_depth`, `crop`, `overlaps_with`, together with free functions `best_image_match`, `distance`, `bool_to_yn` and `coerce_to_numeric`. It closes with:

```
Consider the following guidelines:
- Use base Python (comparison, sorting) for basic logical
  operations,
  left/right/up/down, math, etc.
- Use the llm_query function to access external information and
  answer
  informational questions not concerning the image.
- Return only the function; do not include explanations or example
  usage.

Query: INSERT_QUERY_HERE
```

Two properties are load-bearing. The text nowhere states the required return type, and five of seven examples declare `str`. And the first guideline directs spatial

relations to base Python, excluding the depth API by construction.

The single example returning a patch also fixes the semantics of `compute_depth`:

```
def compute_depth(self):
    """Returns the median depth of the image crop.
    Examples
    -----
    >>> # the bar furthest away
    >>> def execute_command(image) -> ImagePatch:
    >>>     image_patch = ImagePatch(image)
    >>>     bar_patches = image_patch.find("bar")
    >>>     bar_patches.sort(key=lambda bar: bar.compute_depth())
    >>>     return bar_patches[-1]
    """
```

Listing 4.1: The only example in the released prompt that returns an `ImagePatch`, quoted verbatim from `prompts/api.prompt`, lines 100–110.

The example sorts ascending and takes the last element for *furthest away*. The specification therefore requires that a larger value denote a *greater* distance from the camera. Section 5.4 shows the shipped implementation violates this, and that this example is the strongest available evidence for the correction.

4.3.2 C2 — an explicit output contract

C2 appends three sentences and two examples; nothing is removed.

```
This is a VISUAL GROUNDING task. The function must return the
single
ImagePatch that the query refers to. It must NOT return a string,
a bool,
or a list.
Resolve every spatial relation in the query; do not drop it.
```

```
>>> # Query: man on the right
>>> def execute_command(image) -> ImagePatch:
>>>     image_patch = ImagePatch(image)
>>>     man_patches = image_patch.find("man")
>>>     man_patches.sort(key=lambda p: p.horizontal_center)
>>>     return man_patches[-1]

>>> # Query: the cup behind the bottle
>>> def execute_command(image) -> ImagePatch:
>>>     image_patch = ImagePatch(image)
>>>     cup_patches = image_patch.find("cup")
>>>     bottle_patches = image_patch.find("bottle")
>>>     if not cup_patches: return image_patch
```

```

>>>     if not bottle_patches: return cup_patches[0]
>>>     bottle_depth = bottle_patches[0].compute_depth()
>>>     behind = [c for c in cup_patches if c.compute_depth() >
        bottle_depth]
>>>     return behind[0] if behind else cup_patches[0]

```

The clause on not dropping spatial relations is included because the static analysis measured that failure independently of the return type.

4.3.3 C3 — depth primitives

C3 replaces the first guideline with a distinction between the two kinds of spatial relation:

- Use base Python (comparison, sorting) for logical operations and arithmetic.
- Resolve LEFT/RIGHT/ABOVE/BELOW using patch coordinates (left, right, horizontal_center, vertical_center).
- Resolve NEAR/FAR/BEHIND/IN FRONT using the depth functions below. These relations are about distance from the camera and cannot be recovered from image-plane coordinates.

and documents five new functions:

```

def depth_order(patches: List[ImagePatch],
                reverse: bool = False) -> List[ImagePatch]:
    """Returns the patches sorted from NEAREST to FURTHEST from
    the camera.
    Pass reverse=True to sort furthest to nearest."""

def is_behind(a: ImagePatch, b: ImagePatch, margin: float = 0.0)
-> bool:
    """Returns True if a is further from the camera than b."""

def is_in_front_of(a: ImagePatch, b: ImagePatch, margin: float =
0.0) -> bool:
    """Returns True if a is closer to the camera than b."""

def is_between_3d(a: ImagePatch, b: ImagePatch, c: ImagePatch) ->
bool:
    """Returns True if a lies between b and c in depth."""

def distance_3d(a: ImagePatch, b: ImagePatch) -> float:
    """Returns the separation between two patches, combining image
    -plane

```

```
offset and depth. Relative, not metres."
```

Listing 4.2: Signatures presented to the generator in C3 and C4.

`depth_order` is the highest-value addition: a manual audit (Section 7.5) finds *closest*, *nearest*, *farthest* and *closer* account for the majority of depth-word queries, and one ordering primitive covers all four including the ordinal cases (*second closest*) that a pairwise predicate cannot express.

4.3.4 C4 — routing

C4 adds no capability. It states when the capability applies:

- Use the depth functions ONLY when the query actually expresses a depth relation: *closest*, *nearest*, *farthest*, *furthest*, *behind*, *in front of*, *between* (in depth). For every other query they add nothing and cost accuracy.
- If the query is about LEFT/RIGHT/TOP/BOTTOM, sort by patch coordinates and do not call any depth function.
- If the query is about a colour, clothing, size or any other attribute, use `verify_property` or `best_text_match` and do not call any depth function.

and supplies two counter-examples — queries that should *not* invoke the depth API, since the existing examples demonstrate only positive cases:

```
>>> # Query: the man on the left
>>> def execute_command(image) -> ImagePatch:
>>>     image_patch = ImagePatch(image)
>>>     man_patches = image_patch.find("man")
>>>     if not man_patches: return image_patch
>>>     man_patches.sort(key=lambda p: p.horizontal_center)
>>>     return man_patches[0]

>>> # Query: the woman in the red jacket
>>> def execute_command(image) -> ImagePatch:
>>>     image_patch = ImagePatch(image)
>>>     woman_patches = image_patch.find("woman")
>>>     if not woman_patches: return image_patch
>>>     red = [w for w in woman_patches
>>>             if w.verify_property("woman", "red jacket")]
>>>     return red[0] if red else woman_patches[0]
```

C4 exists because C3 was measured and found to trade one failure for another; the sequence is reported in Chapter 8 rather than presented as foresight.

4.4 Evaluation protocol

4.4.1 Metric

The paper’s Table 1 is captioned “we report accuracy on the REC task” while its column header reads “IoU (%)”, and the released evaluator computes mean IoU. These are different quantities. This dissertation reports *accuracy at* $\text{IoU} \geq 0.5$ as primary and mean IoU alongside. On the runs reported here the two differ by roughly one point — C4 on RefCOCO scores 42.00 and 41.81 — so no conclusion depends on the choice, but both are given so comparison is possible in either currency.

4.4.2 Handling of invalid predictions

The released evaluator substitutes the dataset mean box, worth approximately 22.6 IoU, when a program returns nothing; this implementation scores an invalid return as zero. The difference is material for C1: under the paper’s own evaluator, C1 would report approximately 22.6 rather than 0.00. The programs are equally invalid; what differs is whether the evaluator conceals the failure. Scoring zero is the more informative choice for a study whose subject is program validity.

4.4.3 Data

All experiments use testA of RefCOCO and RefCOCO+, with UNC referring expressions and COCO `train2014` images. Each condition is evaluated on the first 500 expressions in file order — deterministic and identical across conditions. This differs from the published evaluation, which reports on the full split after a seeded shuffle; the subset is a compute-driven restriction, recorded as such.

Queries are labelled *spatial* or *non-spatial* by a lexicon of relational terms. The lexicon is a heuristic, not annotation, and Section 7.5 examines its limitations.

Chapter 5

Implementation

5.1 Environment

Two Conda environments are required and never coexist in one interpreter.

Table 5.1: Execution environments. The GLIP fork requires an older PyTorch than the generator; the pipeline is split at the `programs.jsonl` boundary as a result.

Environment	Stage	PyTorch	Notes
vipergpt	generation	2.13.0	transformers 5.x
glip_env	execution	2.1.2+cu121	transformers 4.36.2, NumPy < 2

Hardware is a single NVIDIA H100 NVL (95GB) allocated through SLURM using fractional GPU shards. Compute nodes have no outbound network access, so all model weights and datasets are fetched on the login node and every job asserts offline operation before loading anything.

5.2 Building GLIP for the Hopper architecture

The published environment pins CUDA 11.6, which has no support for compute capability 9.0. The published setup therefore *cannot run at all* on this hardware. Three changes were required.

Removed headers. The `THC/*` headers were deleted from PyTorch after version 1.11. Seven CUDA source files under `maskrcnn_benchmark/csrc/cuda/` referenced them:

```
#include <THC/THCAtomics.cuh>      -> #include <ATen/cuda/Atomic
    .cuh>
#include <THC/THCDeviceUtils.cuh>  -> #include <ATen/cuda/
    CUDAContext.h>
```

`THCCeilDiv`, `THCudaMalloc` and `THCState` were verified absent from live code, so no further rewriting was needed.

Target architectures. `setup.py` was extended with explicit `sm_80` and `sm_90` targets.

NumPy 2.0 aliases. Five Python files used `np.float`, removed in NumPy 1.24, causing construction to fail in `anchor_generator.py` before any inference.

The build must be performed *on a GPU node*: `setup.py` gates `CUDAExtension` on `torch.cuda.is_available()`, so a login-node build silently produces a CPU-only extension with no error.

5.2.1 Independent verification

The compiled kernel was verified before the full pipeline was trusted:

```
boxes = torch.tensor([[0.,0.,10.,10.], [1.,1.,11.,11.],
                     [50.,50.,60.,60.]],
                    device='cuda')
scores = torch.tensor([0.9, 0.8, 0.7], device='cuda')
keep = _C.nms(boxes, scores, 0.5) # -> [0, 2]
```

Listing 5.1: CUDA NMS verified on a hand-checked case. Boxes 0 and 1 overlap above threshold; box 2 is disjoint.

The detector was then validated against RefCOCO ground truth, reaching a mean IoU of 0.776 at threshold 0.2 and 81.5% top-box accuracy on a held-out sample of 200.

5.3 The detector wrapper

A defect in the first wrapper is worth recording because of how it presented. `GLIPDemo` defaults to `tensor_inputs=False`, under which it derives the image size as `original_image.shape` assuming an HWC array. Passing a CHW tensor makes it read $(3, H)$ and rescale every box into a three-pixel-wide image.

```
self._demo = GLIPDemo(
    glip_cfg,
    min_image_size=800,
    confidence_threshold=self.threshold,
    show_mask_heatmaps=False,
    tensor_inputs=True,      # <- selects shape[-2:] and the
                             tensor transform
)
```

Listing 5.2: The one-word correction. Without it, every IoU was *exactly* zero — not low, zero — which is what identified the fault as a coordinate-space bug rather than a detection-quality problem.

Boxes are converted from the detector’s top-left origin to the repository’s bottom-left convention on return, and child patches produced by `find` are offset into their parent’s frame — an error invisible for root-level programs and wrong for any program that crops before finding.

5.4 The depth backend

The shipped wrapper loaded MiDaS through `torch.hub`, which reaches the network and therefore hangs on an offline compute node, and returned the raw prediction. MiDaS predicts *inverse* depth: a larger value means *closer*. Every comparison built on `compute_depth` therefore ran opposite to the semantics the paper’s own example requires.

```
def compute_depth(self, image) -> float:
    """Distance-like depth of the patch: LARGER = FURTHER from the
        camera."""
    m = self._inverse_depth_map(image)
    v = self._center_stat(m)
    if v is None:
        return 0.0
    return 1.0 / max(v, _EPS)
```

Listing 5.3: Corrected depth. The inversion lives here and in `depth_of_region`, and nowhere else; callers are not permitted to read the raw map, which is what produced three separate sign errors.

Two further corrections were required. The DPT processor resizes to a square (384×384), so the depth map does not share the image’s aspect ratio; proportional region reads therefore sampled horizontally squashed regions. The map is now re-sampled to image size. And the depth statistic is taken over the central 50% of a box, because border pixels of a detection are usually background and skew the median.

5.5 The depth primitives

```
def depth_order(patches: list, reverse: bool = False) -> list:
    """Patches sorted NEAR to FAR. reverse=True gives far to near.

    Backs "closest", "nearest", "farthest", "second closest" and
    "third from the front" with a single primitive. Patches whose
    depth
    cannot be computed are dropped rather than silently placed at
    one end.
    """
    if not patches:
        return []
    scored = []
    for p in patches:
        try:
```

```

        d = p.compute_depth()
    except Exception:          # a bad crop must not kill the
        program
        continue
    if d is not None:
        scored.append((p, d))
    scored.sort(key=lambda t: t[1], reverse=reverse)
    return [p for p, _ in scored]

def is_behind(a, b, margin: float = 0.0) -> bool:
    """True if a is further from the camera than b by at least
       margin."""
    return a.compute_depth() > b.compute_depth() + margin

def is_between_3d(a, b, c) -> bool:
    """True if a lies between b and c in depth."""
    lo, hi = sorted([b.compute_depth(), c.compute_depth()])
    return lo < a.compute_depth() < hi

```

Listing 5.4: The contributed primitives, from `pipeline/image_patch.py`. All follow the convention `LARGER = FURTHER`.

The primitives are exposed to generated programs by the executor’s namespace builder, alongside the original API.

5.5.1 Unit tests

Three sign errors occurred during development, none of which was caught by a test. The ordering semantics are now pinned by eighteen assertions over stub patches with known depths, requiring no model and no GPU:

```

near, mid, far = StubPatch("near", 1.0), StubPatch("mid", 5.0),
    StubPatch("far", 9.0)

assert depth_order([far, near, mid]) == [near, mid, far]
assert depth_order([near, far, mid], True) == [far, mid, near]
assert depth_order([far, mid, near])[0] is near # "
    closest"
assert depth_order([near, mid, far])[-1] is far # "
    farthest"
assert depth_order([far, StubPatch("bad", None), near]) == [near,
    far]
assert is_behind(far, near) and not is_behind(near, far)

```

```

assert is_between_3d(mid, near, far)
assert round(distance_3d(a, b), 6) == 5.0          # 3-4-5
    triangle

```

Listing 5.5: Representative assertions from `test_depth_api.py`. A failure here means the sign or the ordering has flipped, and is visible immediately rather than after hours of debugging.

5.6 Execution and scoring

The execution engine runs each program in a restricted namespace with a wall-clock timeout, and returns errors as data rather than raising, so that one bad program does not terminate a 500-sample run.

```

res = execute_program(p["program"], chw, bus, timeout_s=args.
    timeout)
kind = "error" if res.error else classify(res.result)

iou, pred = 0.0, None
if kind == "patch":
    r = res.result
    # ImagePatch is bottom-left origin; undo the flip for COCO
    comparison
    pred = (r.left, H - r.upper, r.right, H - r.lower)
    iou = iou_xyxy(pred, g["gt_xyxy"])
ious.append(iou)          # non-patch returns score 0 -- that is
    the point

```

Listing 5.6: Scoring, from `eval/execute_refcoco.py`. A non-patch return scores zero rather than being excluded — returning a sentence where a box is required is a task failure, and dropping such cases would flatter the baseline.

Because the programs are generated in one environment and executed in another, the two stages are joined by index. The runner asserts that the query string of every program matches the query of the corresponding ground-truth record, and refuses to score if any disagree.

Chapter 6

Experimental Setup

Table 6.1: Experiments. Generation and execution are separate jobs; identifiers are scheduler references retained for provenance.

Experiment	Varies	Holds fixed	Jobs
Static analysis	prompt	model, data	29187, 29193
End-to-end accuracy	prompt	model, modules, data	29290, 29292
Generator scale	model size	prompt, modules, data	29321, 29352, 29404
Seed variance	decoding seed	everything else	29301, 29306, 29526
Depth conditions	prompt (C3, C4)	model, modules, data	29459, 29523–29525
Threshold sweep	detector threshold	held-out sample	29318
Candidate cap	candidate-set size	programs	29563
Cap control	cap $\times$ condition	programs	29565

Decoding is greedy for all headline figures, matching the published temperature of zero. Because greedy decoding is deterministic, variance is estimated separately by sampling at $T = 0.7$ with three seeds; those figures are reported as mean and standard deviation and are never mixed with the greedy values.

6.1 Detector threshold

The paper specifies a detection threshold of 0.5. That value was found to lose eight of twenty detections outright, so a sweep was performed — on RefCOCO/testA samples 200–399, *disjoint* from every sample on which results are reported, so that the threshold is not tuned on the data it is evaluated on.

Table 6.2: Detector threshold sweep, held-out sample of 200. The selected value of 0.2 lies within one point of the optimum for detection.

Threshold	Acc. IoU ≥ 0.5	Mean IoU	No detection	Candidates/image
0.10	81.50	0.7372	21	25.5
0.15	81.50	0.7372	21	25.4
0.20	80.50	0.7336	21	19.7
0.25	70.00	0.6658	21	9.3
0.30	59.50	0.5784	22	5.4
0.40	40.00	0.4350	32	2.4
0.50	31.00	0.3360	56	1.3

fig8_threshold_tradeoff

Figure 6.1: One threshold, two incompatible jobs. The value that maximises top-box detection accuracy also returns roughly twenty candidates per image; a program that must *select* among candidates wants far fewer. The same knob cannot serve both, a tension that recurs in Chapter 8.

Twenty-one of two hundred samples yield no detection at *every* threshold up to 0.4. That floor is not threshold-related: it arises from the head-noun heuristic used to prompt the detector, under which “*player number 8*” reduces to “8” and is discarded.

Chapter 7

Results

7.1 Main result

Table 7.1: Accuracy at $\text{IoU} \geq 0.5$. Qwen2.5-Coder-7B, greedy decoding, $n = 500$ per cell, testA split. C1 is the released prompt; C4 adds both the depth primitives and the rule stating when they apply.

Dataset	Subset	C1	C2	C3	C4
RefCOCO+	overall	0.00	30.80	28.00	37.60
RefCOCO+	spatial ($n=47$)	0.00	4.26	19.15	21.28
RefCOCO+	non-spatial ($n=453$)	0.00	33.55	28.92	39.29
RefCOCO	overall	0.00	39.20	36.40	42.00
RefCOCO	spatial ($n=289$)	0.00	34.26	36.68	37.02
RefCOCO	non-spatial ($n=211$)	0.00	45.97	36.02	48.82

fig1_main_result

Figure 7.1: All four conditions across both datasets and both subsets. The three right-most groups act as controls: an intervention that improved only its target subset would be a trade rather than a gain.

Mean IoU for the same RefCOCO runs is 0.0000, 0.3875, 0.3655 and 0.4181 for C1–C4, tracking accuracy to within about a point throughout.

7.2 Comparison with the published figures

fig16_paper_vs_result

Figure 7.2: Reproduced accuracy against the paper’s Table 1, with the zero-shot baselines it reports alongside. C4 recovers 58% of the RefCOCO figure and 56% of RefCOCO+.

That the perception ceiling exceeds the published figure is informative. The detector available here is good enough to surpass 72.0 if a program selected from it perfectly. The shortfall is therefore in program synthesis, which is measured support for treating the generator substitution (D1) as the dominant term rather than assuming it.

7.3 Why the released prompt scores zero

The result is not a crash. Of 500 programs generated under C1 on RefCOCO, 332 executed to completion and returned a *string*; none returned an `ImagePatch`.

Table 7.2: What each of the 500 samples returned, RefCOCO/testA.

Return type	C1	C2	C4
<code>ImagePatch</code>	0	444	488
<code>str</code>	332	1	0
<code>error</code>	168	55	11

Missing modules are ruled out as a cause. `simple_query` and `llm_query` are given neutral empty-string fallbacks (D10), so no program can fail for lack of a

fig10_vs_paper

Figure 7.3: The dotted line is GLIP alone taking its single best box: 81.5% on a held-out sample of 200 — *above* the paper’s 72.0. Perception is therefore not what separates this reproduction from the original. No decomposition of the gap into individual deviations is shown, because those ablations were not run.

fig7_failure_modes

Figure 7.4: Every sample classified by what happened to it. C1 fails on *return type* and never reaches localisation; by C4 those categories are gone and the residual failure is a localisation miss, which is a perception limit rather than a specification bug. An accuracy figure alone conflates the two.

module that was deliberately not loaded. That concession strictly favours C1 and does not change its score.

7.3.1 A second effect: output format

Under C1 a large fraction of generations never defined `execute_command` at all — the model emitted prose or malformed markdown: 140/500 on RefCOCO and 212/500 on RefCOCO+, against zero under every later condition. The contract disciplines output *format* as well as return type; these are separable failures and are reported separately.

7.3.2 Static analysis

Before execution, the programs were analysed by parsing.

Table 7.3: Static analysis of generated programs, $n = 500$ per cell. “Task-invalid” denotes a program whose `return` yields a string rather than a patch; “spatial dropped” denotes a spatial query whose program contains no positional operation at all.

Metric	C1 RefCOCO	C2 RefCOCO	C1 RefCOCO+	C2 RefCOCO+
Task-invalid return	98.8%	0.2%	98.2%	31.9%
Spatial constraint dropped	17.3%	1.7%	99.6%	2.89
Parse rate	99.8%	96.6%		
Mean API calls per program	2.45	1.66		

The contract costs parse rate — 3.2 points on RefCOCO and 7.4 on RefCOCO+ — because a stricter requirement induces a small rate of syntactically invalid generations. Net structural validity nonetheless rises from roughly 1% to 96%.

7.4 Generator scale

Table 7.4: Generator scale, condition C2, greedy, RefCOCO/testA, $n = 500$.

Generator	C1 acc.	C2 acc.	spatial	non-spatial	C1 task-invalid
Qwen2.5-Coder-1.5B	0.80	34.40	34.95	33.65	95.8%
Qwen2.5-Coder-7B	0.00	39.20	34.26	45.97	98.8%
Qwen2.5-Coder-32B	0.00	40.80	34.95	48.82	87.6%

Two conclusions follow. The specification gap is scale-independent: a 32B generator still emits 87.6% task-invalid programs under C1 and still scores 0.00, and the same three sentences repair it at every size. Scaling 1.5B $\rightarrow$ 32B buys 6.4 accuracy points; the contract buys 33 to 41.

Figure 7.5: Non-spatial accuracy climbs with capacity; spatial accuracy does not move. Same images, same detector, same prompt — only the generator differs. This is the evidence that the spatial deficit is an interface problem rather than a capacity one.

More consequentially, **spatial accuracy does not respond to scale at all** — 34.95/34.26/34.95, with the smallest and largest models identical to two decimal places — while non-spatial accuracy rises by 15.2 points. A larger generator writes better programs for everything except spatial relations, where it writes programs that are exactly as wrong. If the deficit were a program-synthesis problem, capacity would help; it demonstrably does elsewhere.

An incidental observation: under C1, the 32B model failed to define `execute_command` in 304/500 generations against 16/500 for 1.5B. The larger instruction-tuned model is more prone to answering in prose when the prompt does not demand a function, which is why 32B scores 0.00 where 1.5B manages 0.80.

7.5 Spatial breakdown

We expected the spatial gap to *shrink* on RefCOCO+, which forbids location words. It widened threefold: 9.3 points on RefCOCO against 28.1 on RefCOCO+ under C2.

A manual audit of all 47 RefCOCO+ spatial queries explains why. The two subsets contain different kinds of spatial language.

Forbidding location words removed viewer-frame *directional* expressions and left depth-ordering and relational ones. The released prompt’s instruction to use base Python for “left/right/up/down” works for the first and cannot work for the second. Consistently, the call-frequency analysis finds `compute_depth` invoked 29 times against 547 for `find`: the geometric API is effectively unused, by design of the prompt.

Two further observations support this reading. All three successes in the RefCOCO+ spatial subset under C2 are depth queries with very high IoU (0.97, 0.96,

Table 7.5: The two spatial subsets are not the same kind of query.

	RefCOCO	RefCOCO+
Type	2D image-plane	depth and inter-object
Examples	<i>man on right, person bottom left</i>	<i>man closest to us, farthest man, umpire behind catcher</i>
Resolvable by	sorting patch coordinates in base Python	requires <code>compute_depth</code> or 3D geometry
Accuracy (C2)	34.26	4.26

0.92): when depth ordering is attempted correctly the localisation is excellent, but it is usually not attempted. And across seeds, spatial variance is roughly four times non-spatial variance, reported in Section 7.8.

Caveat. $n = 47$, so 4.26% is 2 of 47 and 21.28% is 10 of 47. The lexicon also produces roughly five false positives in that subset — “*writing on back*”, “*long sleeved*” matching *side* — which would push true depth-query accuracy lower still. The direction is established; the value is not.

7.6 API adoption

The principal risk in extending an API is that the generator ignores the extension, as it largely ignores `compute_depth`. It did not.

Table 7.6: API calls across 500 generated programs, RefCOCO/testA.

Condition	find	compute_depth	depth_order	other depth primitives
C1	548	29	0	0
C2	639	23	0	0
C3	598	19	119	63
C4	559	—	37	7

7.7 Providing a capability is not sufficient

C3 offers the primitives; C4 adds three sentences saying when they apply. The difference between them is the sharper finding of this dissertation.

C3 wins its target subset — RefCOCO+ depth queries move $4.26 \rightarrow 19.15$ — and loses elsewhere: non-spatial accuracy falls $45.97 \rightarrow 36.02$ on RefCOCO and $33.55 \rightarrow 28.92$ on RefCOCO+, so overall accuracy drops. `depth_order` is invoked 119 times on RefCOCO, where most queries need no depth at all. *Given a new primitive, the generator reaches for it indiscriminately.*

Figure 7.6: Zero adoption under the older prompts, strong adoption once the primitives are offered. Under C4 the count falls to 37 — the routing rule working, not the primitive being abandoned, since RefCOCO contains few depth queries. On RefCOCO+, which has 47 depth-word queries, C4 makes 33 `depth_order` calls against 167 under C3.

C4 improves every cell of both datasets. The depth gain survives (21.28) while non-spatial rises above C2 as well (48.82 against 45.97), presumably because the counter-examples also clarify the 2D and attribute paths. Execution robustness improves monotonically: RefCOCO errors $55 \rightarrow 24 \rightarrow 11$ and patch returns $444 \rightarrow 475 \rightarrow 488$ for $C2 \rightarrow C3 \rightarrow C4$.

This is the same class of omission as the first result. In C1 the specification failed to state *what* to return; in C3 it failed to state *when* a capability applies. Both are failures of the contract rather than of the model.

7.8 Seed variance

Greedy decoding is deterministic, so variance was estimated by sampling at $T = 0.7$ with three seeds.

Table 7.7: Seed variance, RefCOCO/testA, $T = 0.7$, seeds 1–3, $n = 500$.

Condition	overall	spatial	non-spatial
C2 contract	36.07 ± 1.75	32.87 ± 3.35	40.44 ± 0.72
C3 + depth	34.40 ± 0.40	34.02 ± 0.80	34.92 ± 0.55
C4 + routing	42.07 ± 0.61	38.29 ± 0.72	47.23 ± 1.19

Two observations. First, **greedy decoding understates C4**. C2’s greedy result (39.20) sits 1.8 standard deviations *above* its own sampled mean (36.07) — greedy was fortunate for C2 — while C4’s greedy (42.00) and sampled (42.07) results coincide. The honest comparison is 6.0 points, not the 2.8 the greedy table suggests.

Second, and unanticipated: **the primitives buy stability as well as accu-**

fig4_qualitative

Figure 7.7: Depth queries where C4 succeeds and C2 fails. Red: C2's prediction. Teal: C4's. Dashed: ground truth. The failures are not near-misses — without a primitive for *closest*, the program selects a different object entirely.

fig6_seed_variance

Figure 7.8: Left: C4 separates from C2 by 6.0 points against standard deviations under two. Right: standard deviation on spatial queries alone. The explicit primitive removes the improvisation the generator otherwise performs afresh on every sample.

racy. Spatial-query standard deviation falls from ± 3.35 under C2 to ± 0.72 under C4, a factor of 4.6. Under C2 the generator improvises spatial logic on every sample — sometimes sorting by coordinate, sometimes dropping the relation entirely — and which it does depends on the seed. Once `depth_order` exists there is one way to express the relation and the variance collapses. An explicit primitive makes the system’s behaviour reproducible, not merely more accurate.

7.9 Robustness of the metric

fig11_iou_sweep

Figure 7.9: Accuracy recomputed at every IoU threshold from 0.3 to 0.9, and the C4–C2 difference. The threshold of 0.5 is used because the paper uses it, but the C4 advantage stays positive across the whole range.

Were the advantage confined to the neighbourhood of 0.5, it would suggest the conditions differ in box tightness rather than in object selection. They do not.

fig5_iou_distribution

Figure 7.10: Left: distribution over samples scoring anything at all. Right: share of samples scoring exactly zero. The distributions are strongly bimodal — programs either localise well or miss entirely — which is what one expects when the failure mode is selecting the wrong object rather than drawing a loose box.

Chapter 8

Rejected Hypotheses and Controls

Three hypotheses were tested and rejected, and one intervention was disqualified by a control. They are reported because a reproduction study in which nothing failed would be less credible, and because in two cases the rejected result was larger than the accepted one.

8.1 Rejection 1: the collapse hypothesis

Milestone 1 was designed to test a specific claim: that spatial-relational queries would cause the generator to collapse the whole query into a single opaque `simple_query` call. The paper’s GQA prompt instructs exactly this, listing relational terms and directing the model to ask directly rather than decompose.

The measured collapse rate was approximately 0% — one program in 289 spatial queries.

The hypothesis was well-posed and aimed at the wrong task. That instruction applies to GQA, not to grounding, and grounding cannot use `simple_query` at all because it must return a patch. The cost of establishing this was a single 63-minute job, incurred before any perception model had been built — which is why the generation stage was studied first.

The same run produced the return-type finding that became the first result of this dissertation.

8.2 Rejection 2: RefCOCO+ as a clean control

RefCOCO+ forbids location words, so it was expected to serve as a control in which the spatial/non-spatial gap would narrow or vanish. It widened threefold (Section 7.5).

The audit that followed produced the distinction between image-plane and depth-based spatial language, which is the entire justification for Milestone 3. A rejected hypothesis produced the contribution.

8.3 Rejection 3: the primitives alone

C3 was expected to suffice. It improved its target subset and degraded the others, failing criterion C-2, and was superseded by C4 (Section 4.3).

8.4 A fifteen-point result, disqualified

The detector returns roughly 20 candidates per image at the selected threshold. Capping `find` to k of them, on the *same* C4 programs:

Table 8.1: Candidate cap applied to C4 programs, RefCOCO/testA.

Cap	overall	spatial	non-spatial
off	42.80	37.72	49.76
$k = 12$	45.20	41.52	50.24
$k = 6$	49.60	47.75	52.13
$k = 3$	57.60	61.59	52.13

57.60 would be the best figure in this project by fifteen points, and 61.59 on spatial queries would exceed every non-spatial result obtained. It is not reportable, and the control is why.

Applying the identical cap to **C2**, which contains no depth primitives at all:

Table 8.2: The control required by criterion C-1. C2 has no depth reasoning anywhere in the pipeline.

	no cap	$k = 3$	gain
C2 spatial	34.60	57.09	+22.5
C4 spatial	37.72	61.59	+23.9
C2 non-spatial	45.02	45.50	+0.5
C4 non-spatial	49.76	52.13	+2.4

C2’s spatial accuracy rises 22.5 points from the cap alone. Of C4’s 23.9, roughly 1.4 points are attributable to depth and the rest to the candidate set.

The mechanism is arithmetic. Choosing among three candidates rather than twenty raises the hit rate by construction — random selection alone scores 33% at $k = 3$ against 5% at $k = 20$ — and in RefCOCO the referred object is almost always among the most confident detections. That non-spatial queries gain almost nothing (+0.5 for C2) confirms it: those queries verify attributes rather than selecting among same-class candidates, so a smaller set does not help them. Only *pick one of N people* benefits, and it benefits however the pick is made.

Decision. `max_detections` remains disabled for every reported number. Non-maximum suppression is likewise left off: it helps RefCOCO marginally (42.00 →

fig13_cap_control

Figure 8.1: Left: both conditions with and without a three-candidate cap. Right: the gain the cap alone provides, by subset. C2 gains almost exactly what C4 gains, so the improvement belongs to the candidate set rather than to depth reasoning. Non-spatial queries barely move, which identifies the mechanism.

42.80) but harms RefCOCO+ depth queries badly ($21.28 \rightarrow 12.77$), because suppressing overlapping boxes removes exactly the depth-distinct instances a depth query must tell apart.

What survives. C4’s advantage over C2 holds under both settings — 3.8 points uncapped, 5.4 capped — so no claim in this work rests on the cap.

8.5 A sign error that a synthetic test endorsed

The depth sign was established, reversed, and re-established during development. The reason is worth recording.

The initial implementation returned the raw MiDaS prediction. Believing this to be inverse depth, an inversion was applied. A synthetic test scene — a brightness gradient with a large dark object low in frame — then indicated the inversion was wrong, and it was removed. That scene was invalid: the model read the brightness gradient as depth.

The decisive experiment removes both the detector and the synthetic data. Take RefCOCO+ depth queries, use **ground-truth boxes only**, and ask whether the annotated target sits at the correct end of the depth ordering.

Two lessons are recorded. **Synthetic scenes are unreliable for validating perceptual signals**, because the model latches onto whatever cue the synthetic im-

fig12_headroom

Figure 8.2: Depth-ordering selection study. Grey: the correct box was present among the candidates. Teal: it was selected. The gap between bars is an ordering failure; the gap from grey to 100% is a detection failure. The binding constraint has moved from ordering to detection.

Table 8.3: Depth sign validated against ground truth, no detector involved, $n = 25$.

Target at correct depth extreme	
without inversion	2/25 = 8%
with inversion	20/25 = 80%
chance (3.8 objects per image)	26%

fig9_depth_sign

Figure 8.3: 8% is far *below* the chance level of 26%. A weak signal scores near chance; an inverted one scores below it. That distinction is what identified the fault.

age actually contains. And the paper’s own `compute_depth` example — `sort(...); return bar_patches[-1]` for *furthest away* — independently requires the corrected sign, so the correction is not merely empirically supported but required by the specification being reproduced.

Chapter 9

Deviations from the Published Configuration

Every departure is recorded with its reason and expected effect, per criterion C-5.

Table 9.1: Deviation register.

ID	Deviation	Reason and expected effect
D1	Codex $\rightarrow$ Qwen2.5-Coder	Codex is retired; no faithful option exists. This substitution is the subject of the study and the dominant term in the gap to 72.0.
D2	Scope restricted to grounding	The four-table configuration requires roughly 250 GB against 231 GB of quota; NExT-QA alone needs over 100 GB of video, and OK-VQA depends on GPT-3, also retired. No cross-task claim is made.
D3	RefCOCO annotations from the Internet Archive	The canonical host no longer resolves. The archived files are byte-identical. No effect.
D4	Dependency versions	The environment pre-existed, so the pinned versions were not applied. Frozen to a lockfile. No effect on the text-only path.
D5	GLIP built for sm_90	CUDA 11.6 has no support for compute capability 9.0, so the published environment cannot run on this hardware. Headers updated, target architectures added; kernels unchanged. Verified numerically.
D6	NumPy 2.0 aliases in GLIP	Five files used <code>np.float</code> , removed in NumPy 1.24. Semantics unchanged.
D7	Offline operation	GLIP fetches a tokenizer at load time; compute nodes have no network. Cached in advance and asserted before every job.
D8	Detector threshold 0.5 $\rightarrow$ 0.2	The paper’s value loses 8 of 20 detections. Reswept on a held-out slice ($n=200$, disjoint from all reported cells): 0.2 is within 1.0 point of optimal for detection. Material and now validated.

ID	Deviation	Reason and expected effect
D9	X-VLM → CLIP ViT-L	X-VLM weights are distributed via a file-sharing link and cannot be fetched by script. CLIP scores globally rather than at region level, so attribute verification should be weaker; costliest on Ref-COCO+, whose expressions are attribute-based.
D10	<code>simple_query</code> , <code>llm_query</code> return ""	Loading BLIP-2-XXL and an 8B text model is out of scope. Without a fallback, a program calling them raises, making its failure attributable to our configuration. Strictly favours C1; no effect on later conditions.
D11	Depth backend corrected	The shipped wrapper loaded MiDaS over the network and returned inverse depth unmodified, so comparisons ran opposite to the documented semantics. Now loaded offline, resampled to image size, and inverted. Every depth number depends on this; pinned by 18 unit assertions.
D12	Detection scores exposed; candidate controls added	<code>find</code> now also returns per-box confidence, with optional <code>max_detections</code> and NMS. Both disabled by default : enabling them raises accuracy by 15 points, but a control shows the gain belongs to the candidate set rather than to the method (Chapter 8). Recorded as a negative result.

9.1 Undocumented at the time of writing

Two further differences were identified during the audit of this dissertation and are recorded here for completeness.

The `crop_larger_margin` option, which expands detections by 10%, is enabled in the published configuration and disabled in the evaluation configurations used here. Its effect has not been measured.

The published evaluator shuffles the split under a fixed seed and reports on the full testA set; this work takes the first 500 expressions in file order. Both are deterministic, but they are not the same 500 samples.

Chapter 10

Limitations

1. **The reproduction is not faithful.** 42.00 against a reported 72.0. The contributors are enumerated in Chapter 9 but not decomposed quantitatively, because the per-deviation ablations were not run.
2. **Metric and fallback differ from the published evaluator.** Accuracy at $\text{IoU} \geq 0.5$ rather than mean IoU, and zero rather than a mean-box substitute for an invalid prediction. Both figures are reported wherever a comparison is drawn.
3. **The depth subset is small.** $n = 47$; 21.28% is 10 of 47. The direction is established, the value is not. The lexicon used to identify spatial queries is a heuristic with an estimated five false positives in that subset.
4. **Seed coverage is asymmetric.** Seeds were run for RefCOCO only; every RefCOCO+ figure, including the headline 21.28, is single-run.
5. **X-VLM was never built.** CLIP is a substitute whose weakness is expected to fall most heavily on RefCOCO+.
6. **Evaluation is on a 500-sample subset** of testA rather than the full split.
7. **The candidate-set tension is unresolved.** The detector threshold is correct for detection and wrong for selection; `max_detections` addresses this but was disqualified as a contribution and disabled.
8. **Provenance is by job identifier rather than commit hash** for the execution runs, because `git` is not installed on the compute nodes. The job scripts are committed, so the configuration is recoverable, but the automatic stamping intended by the design is absent.

Chapter 11

Conclusion and Future Work

11.1 Conclusion

This dissertation set out to reproduce ViperGPT after the withdrawal of its program generator, and to determine which of its reported behaviour belonged to the method rather than to that generator.

The answer to RQ1 is that the released specification carries an unstated output contract. With an open-weights generator, 98.8% of programs return a natural-language string where a bounding box is required, and executed accuracy is 0.00 — not because the system crashes, but because 332 of 500 programs run to completion and answer the wrong kind of question. Three sentences stating the contract raise accuracy to 39.20.

The answer to RQ2 is that this is a property of the specification, not of capacity. The failure occurs at 1.5B, 7B and 32B parameters and is repaired at each. Scaling the generator twentyfold buys approximately six accuracy points where stating the contract buys thirty-three to forty-one.

The answer to RQ3 is that accuracy differs sharply by the kind of spatial language. Across the same three model sizes, non-spatial accuracy rises $33.65 \rightarrow 48.82$ while spatial accuracy remains flat at 34.95/34.26/34.95. A manual audit shows that RefCOCO’s spatial queries are image-plane and RefCOCO+’s are depth-based, and that the latter score 4.26 against the former’s 34.26. The API exposes one scalar for depth and the prompt directs the generator away from geometry; the deficit is in the interface.

The answer to RQ4 is qualified and is the more interesting result. Extending the API with depth-ordering primitives improves depth queries from 4.26 to 19.15 but costs ten points of non-spatial accuracy: the generator applies a new primitive indiscriminately. Adding three sentences that delimit when the primitives apply improves *every* subset of both datasets, reaching 21.28 on depth queries and 42.00 overall (42.07 ± 0.61 over three seeds against 36.07 ± 1.75), and reduces the seed-to-seed variance of spatial queries by a factor of 4.6.

The two results share a form. In the first, the specification failed to say *what* a program must return. In the second, it failed to say *when* a capability applies. Neither is a limitation of the model, and neither is visible until the model that had

been supplying the missing constraint is replaced.

11.2 Future work

Complete the substitution register. Building X-VLM (D9) would close the deviation most likely to be costing accuracy on RefCOCO+, and would allow the attribute path to be attributed rather than assumed.

Resolve the candidate-set tension. The detector threshold is tuned for detection and is wrong for selection. A principled treatment — a query-conditional candidate budget, or a selection-aware detector — would address a constraint that currently limits both depth ordering and attribute checking, and would do so without the dataset artefact that disqualified the naive cap.

Extend the seed protocol to RefCOCO+ and to the depth subset specifically, where the present figures are single-run and small.

Metric depth. The primitives claim ordering only, because the datasets carry no camera intrinsics. A dataset with intrinsics, or a metric depth model, would allow `position_3d` and `physical_size` — implemented but unused — to be evaluated, and would extend the API to relations that ordering cannot express.

Generalise the specification finding. The result here concerns one prompt and one task. Whether other program-synthesis systems carry comparable unstated contracts is testable by the same method: substitute the generator and measure what breaks.

Bibliography

- [1] Jacob Andreas, Marcus Rohrbach, Trevor Darrell, and Dan Klein. Neural module networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [2] Samuel J. Bell and Onno P. Kampman. Perspectives on machine learning reproducibility. *arXiv preprint arXiv:2109.06827*, 2021.
- [3] Boyuan Chen, Zhuo Xu, Sean Kirmani, Brian Ichter, Danny Driess, Pete Florence, Dorsa Sadigh, Leonidas Guibas, and Fei Xia. SpatialVLM: Endowing vision-language models with spatial reasoning capabilities. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [4] Tanmay Gupta and Aniruddha Kembhavi. Visual programming: Compositional visual reasoning without training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [5] Ronghang Hu, Jacob Andreas, Trevor Darrell, and Kate Saenko. Explainable neural computation via stack neural module networks. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018.
- [6] Aishwarya Kamath, Mannat Singh, Yann LeCun, Gabriel Synnaeve, Ishan Misra, and Nicolas Carion. MDETR: Modulated detection for end-to-end multi-modal understanding. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [7] Amita Kamath, Jack Hessel, and Kai-Wei Chang. What’s “up” with vision-language models? investigating their struggle with spatial reasoning. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [8] Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 2023.
- [9] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2023.

- [10] Liunian Harold Li, Pengchuan Zhang, Haotian Zhang, Jianwei Yang, Chunyuan Li, Yiwu Zhong, Lijuan Wang, Lu Yuan, Lei Zhang, Jenq-Neng Hwang, Kai-Wei Chang, and Jianfeng Gao. Grounded language-image pre-training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [11] Fangyu Liu, Guy Emerson, and Nigel Collier. Visual spatial reasoning. *Transactions of the Association for Computational Linguistics*, 2023.
- [12] Pan Lu, Baolin Peng, Hao Cheng, Michel Galley, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, and Jianfeng Gao. Chameleon: Plug-and-play compositional reasoning with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [13] Matthias Minderer, Alexey Gritsenko, Austin Stone, Maxim Neumann, Dirk Weissenborn, Alexey Dosovitskiy, Aravindh Mahendran, Anurag Arnab, Mostafa Dehghani, Zhuoran Shen, Xiao Wang, Xiaohua Zhai, Thomas Kipf, and Neil Houlsby. Simple open-vocabulary object detection with vision transformers. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2022.
- [14] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2021.
- [15] René Ranftl, Katrin Lasinger, David Hafner, Konrad Schindler, and Vladlen Koltun. Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.
- [16] Sanjay Subramanian, William Merrill, Trevor Darrell, Matt Gardner, Sameer Singh, and Anna Rohrbach. ReCLIP: A strong zero-shot baseline for referring expression comprehension. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.
- [17] Sanjay Subramanian, Medhini Narasimhan, Kushal Khangaonkar, Kevin Yang, Arsha Nagrani, Cordelia Schmid, Andy Zeng, Trevor Darrell, and Dan Klein. Modular visual question answering via code generation. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

- [18] Dídac Surís, Sachit Menon, and Carl Vondrick. ViperGPT: Visual inference via python execution for reasoning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [19] Kexin Yi, Jiajun Wu, Chuang Gan, Antonio Torralba, Pushmeet Kohli, and Joshua B. Tenenbaum. Neural-symbolic VQA: Disentangling reasoning from vision and language understanding. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [20] Licheng Yu, Patrick Poirson, Shi Yang, Alexander C. Berg, and Tamara L. Berg. Modeling context in referring expressions. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2016.
- [21] Yan Zeng, Xinsong Zhang, and Hang Li. Multi-grained vision language pre-training: Aligning texts with visual concepts. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2022.